

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа 3

Выполнила:
Яковенко К. А.

Группа:
К3340

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

В данной домашней работе требовалось реализовать тестирование API средствами Postman, написать тесты внутри Postman и получить один комплексный сценарий по основному процессу приложения. В качестве тестируемой системы используется REST API платформы для фитнес-тренировок и здоровья.

Ход работы

Подготовка тестового окружения

Перед началом тестирования backend-приложение было запущено локально. Базовый адрес API был задан как `http://localhost:8000/api`. Для тестирования была создана коллекция Postman «HW3 Fitness API Main Scenario» и окружение «Fitness Local».

Окружение использовалось для хранения значений, которые нужны нескольким запросам. Например, после авторизации JWT-токен сохраняется в переменную `accessToken`, а после получения списка тренировок сохраняется `workoutId` выбранной тренировки.

Переменная	Назначение
<code>baseUrl</code>	Базовый адрес API: <code>http://localhost:8000/api</code>
<code>testEmail</code>	Email тестового пользователя. Генерируется автоматически перед регистрацией.
<code>testPassword</code>	Пароль тестового пользователя.
<code>accessToken</code>	JWT-токен, полученный после авторизации.
<code>workoutId</code>	ID выбранной тренировки из списка.
<code>trainingPlanId</code>	ID выбранного тренировочного плана.
<code>userTrainingPlanId</code>	ID записи пользователя на тренировочный план.
<code>bodyMetricId</code>	ID созданного замера тела.

Разработка сценария

Мной был разработан следующий сценарий:

№	Запрос	Метод	Endpoint	Назначение
1	Регистрация нового пользователя	POST	/auth/register	Создание пользователя и профиля
2	Авторизация пользователя	POST	/auth/login	Получение JWT-токена
3	Получение данных текущего пользователя	GET	/users/me	Проверка авторизации
4	Получение каталога тренировок	GET	/workouts	Получение списка тренировок и сохранение workoutId
5	Фильтрация тренировок по длительности	GET	/workouts?durationMinFrom=10&durationMinTo=40	Проверка работы фильтра
6	Просмотр выбранной тренировки	GET	/workouts/{workoutId}	Получение деталей тренировки
7	Получение каталога тренировочных планов	GET	/training-plans	Получение списка планов и сохранение trainingPlanId
8	Запись пользователя на тренировочный план	POST	/training-plans/{trainingPlanId}/enroll	Создание пользовательского прохождения плана
9	Получение списка планов текущего пользователя	GET	/users/me/training-plans	Проверка записи на план
10	Обновление прогресса прохождения плана	PATCH	/users/me/training-plans/{userTrainingPlanId}	Изменение progressPercent
11	Добавление записи о замерах тела	POST	/users/me/body-metrics	Фиксация пользовательского прогресса
12	Отмена участия в тренировочном плане	PATCH	/users/me/training-plans/{userTrainingPlanId}	Cleanup: перевод записи в статус cancelled
13	Выход пользователя из системы	POST	/auth/logout	Отзыв JWT-токена
14	Проверка недействительности токена после выхода	GET	/users/me	Проверка, что старый токен больше не работает

Запросы расположены именно в таком порядке, потому что часть шагов зависит от данных, полученных ранее. Например, сначала выполняется вход в систему и сохраняется accessToken, затем этот токен используется в защищённых запросах. Аналогично после получения списка тренировок сохраняется workoutId, а после получения списка планов - trainingPlanId.

Добавление автоматических тестов

Для каждого запроса были добавлены автоматические тесты во вкладке Post-response. Тесты проверяют статус ответа, наличие обязательных полей и корректность данных. Это позволяет запускать весь сценарий как автоматическую проверку, а не выполнять запросы только вручную.

Пример теста для авторизации

В запросе авторизации проверяется успешный статус ответа и наличие accessToken. Полученный токен сохраняется в переменную окружения, чтобы его могли использовать следующие запросы.

```
Pre-request
Post-response ●

1  pm.test("Login: status is 200", function () {
2      pm.response.to.have.status(200);
3  });
4
5  const jsonData = pm.response.json();
6
7  pm.test("Login: response has accessToken", function () {
8      pm.expect(jsonData).to.have.property("accessToken");
9      pm.expect(jsonData.accessToken).to.be.a("string");
10 });
11
12 pm.environment.set("accessToken", jsonData.accessToken);
```

Пример теста фильтрации тренировок

Для проверки фильтрации по длительности тестируется не только статус ответа, но и содержимое массива items. Каждая тренировка из ответа должна находиться в заданном диапазоне длительности от 10 до 40 минут.

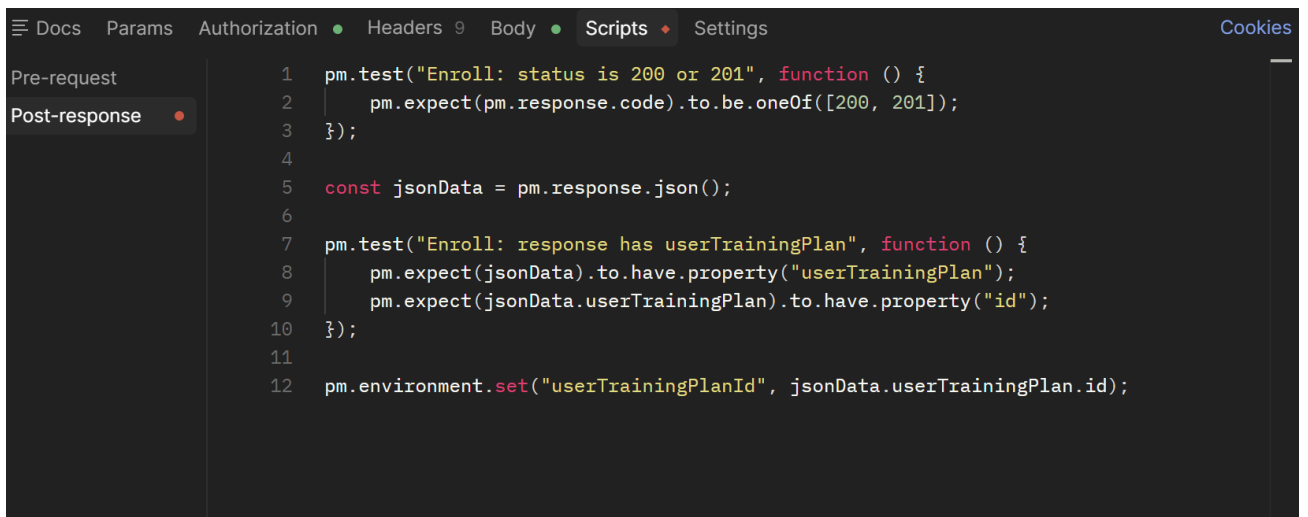
```
Pre-request
Post-response ●

1  pm.test("Filtered workouts: status is 200", function () {
2      pm.response.to.have.status(200);
3  });
4
5  const jsonData = pm.response.json();
6
7  pm.test("Filtered workouts: items array exists", function () {
8      pm.expect(jsonData.items).to.be.an("array");
9  });
10
11 pm.test("Filtered workouts: every item is within duration range", function () {
12     jsonData.items.forEach((item) => {
13         pm.expect(item.durationMin).to.be.at.least(10);
14         pm.expect(item.durationMin).to.be.at.most(40);
15     });
16 });
```

Проверка записи на тренировочный план и cleanup

После получения списка тренировочных планов сценарий записывает пользователя на выбранный план. В ответе сохраняется `userTrainingPlanId` - идентификатор записи пользователя на план. Затем этот идентификатор используется для обновления прогресса и отмены участия в плане.

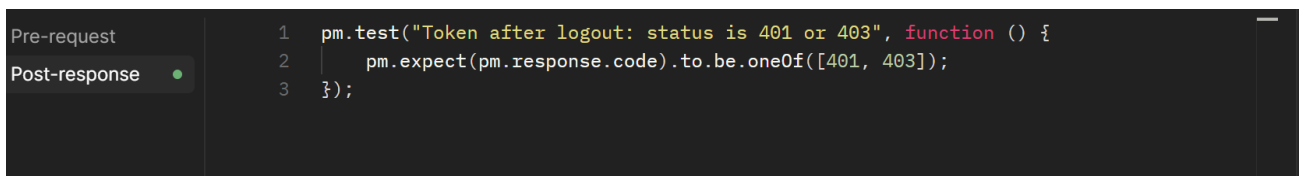
Шаг отмены прохождения плана добавлен как `cleanup`. Он переводит запись в статус `cancelled`. Это удобнее, чем физически удалять запись, потому что история действий пользователя сохраняется, но активное прохождение завершается.



```
1 pm.test("Enroll: status is 200 or 201", function () {
2   pm.expect(pm.response.code).to.be.oneOf([200, 201]);
3 });
4
5 const jsonData = pm.response.json();
6
7 pm.test("Enroll: response has userTrainingPlan", function () {
8   pm.expect(jsonData).to.have.property("userTrainingPlan");
9   pm.expect(jsonData.userTrainingPlan).to.have.property("id");
10 });
11
12 pm.environment.set("userTrainingPlanId", jsonData.userTrainingPlan.id);
```

Проверка logout

В конце сценария выполняется выход пользователя из системы. После `logout` токен должен быть отозван. Для проверки этого поведения выполняется дополнительный запрос к защищённому endpoint `/users/me` с тем же токеном. Ожидаемый результат - статус 401 или 403.



```
1 pm.test("Token after logout: status is 401 or 403", function () {
2   pm.expect(pm.response.code).to.be.oneOf([401, 403]);
3 });
```

Результаты выполнения коллекции

После настройки всех запросов коллекция была запущена целиком через Collection Runner.

Source	Environment	Iterations	Duration	All tests
Runner	Fitness	1	1s 741ms	35
RUN SUMMARY				
				1
> POST	Register User		3 0	
> POST	Login User		2 0	
> GET	Get Current User		3 0	
> GET	Get Workouts		3 0	
> GET	Filter Workouts by Duration		3 0	
> GET	Get Workout Details		3 0	
> GET	Get Training Plans		3 0	
> POST	Enroll Training Plan		2 0	
> GET	Get My Training Plans		3 0	
> PATCH	Update training Plan Progress		2 0	
> POST	Create Body Metric		2 0	
> PATCH	Cancel Training Plan Enrollment		3 0	
> POST	Logout		2 0	
> GET	Token Check After Logout		1 0	

Вывод

Выполнив данную работу, я познакомилась с системой тестирования Postman и получила базовые навыки автоматического тестирования.